TP # 5

Boucle `for` : Sommes et produits

Calculs de sommes et produits simples

- Pour calculer une somme $\sum_{k=m}^{n} a_k$:

```
S = 0 # avant la boucle, S = 0 ("somme vide")

for k in range(m,n+1) : # boucle pour k variant entre m et n
    S = S + a_k # on ajoute le k-eme terme

print(S) / return(S) # on affiche / on renvoie la valeur finale
```

- Pour calculer un produit $\prod_{k=m}^{n} a_k$:

```
P = 1 # avant la boucle, P = 1 ("produit vide")

for k in range(m,n+1) : # boucle pour k variant entre m et n
    P = P * a_k # on multiplie par le k-eme terme

print(P) / return(P) # on affiche / on renvoie la valeur finale
```

Exercice 1

1. Compléter le programme suivant pour afficher la valeur de la somme $\sum_{k=0}^{100}(2k+1)$.

```
S = 0

for k in range(0,101) : # k = 0, 1, ..., 100

        S = S + 2*k + 1
        # ou : S += 2*k + 1

print(S)
```

2. Calculer cette somme (avec des maths !) et vérifier le résultat annoncé par Python.

$$\sum_{k=0}^{100}(2k+1) = 2\sum_{k=0}^{100} k + \sum_{k=0}^{100} 1 = 2 \times \frac{100 \times 101}{2} + 101 = 101 \times 101 = 10201.$$

Exercice 2

1. Compléter la définition de la fonction `factorielle`.

```
def factorielle(n) :

        P = 1

        for k in range(1,n+1) : # k = 1, 2, ..., n

            P = P * k
            # ou : P *= k

        return P
```

Remarque : Si on teste ce programme avec $n = 0$, l'instruction `range(1,1)` correspond à la liste vide en Python. On fera donc 0 passages de boucle : tout se passe comme si on "ignorait" la boucle. Le programme effectue donc `P = 1` puis directement `return P`. On renvoie donc bien la valeur $1 = 0!$.

2. Tester cette fonction pour calculer $10! = 3\,628\,800$

Exercice 3

Pour tout $n \in \mathbb{N}^*$, soit $S_n = \sum_{i=1}^{n} \frac{1}{i^2}$. On veut montrer : $\lim_{n\to+\infty} S_n = \frac{\pi^2}{6}$.

1. Définir une fonction `somme` qui prend en entrée $n \in \mathbb{N}^*$ et renvoie la valeur S_n.

```
def somme(n) :
    S = 0
    for i in range(1,n+1) :
            S = S + 1/i**2
    return S
```

2. A l'aide de Python, compléter avec des valeurs approchées :

$\frac{\pi^2}{6} \simeq 1,64493$ (on tape `import numpy as np` puis `np.pi**2/6` dans la console)

$$S_1 = 1, \quad S_{10} \simeq 1,5497\,, \quad S_{100} \simeq 1,6349\,, \quad S_{10000} \simeq 1,6448$$

Remarque 1

Pour renvoyer la valeur finale :

- A l'intérieur d'une fonction on utilisera systématiquement `return`.
- En dehors d'une fonction (comme dans l'exercice 1), on utilisera plutôt `print`.

Sommes doubles

- Pour calculer une somme double $\sum_{i=m}^{n}\left(\sum_{j=q}^{p} a_{i,j}\right)$, on utilise deux boucle "imbriquées" :

```
S = 0 # avant la boucle, S = 0 ("somme vide")

for i in range(m,n+1) : # boucle pour i variant entre m et n
    for j in range(q,p+1) : # boucle pour j variant entre q et p
        S = S + a_(i,j) # on ajoute le terme correspondant

print(S) / return(S) # on affiche / on renvoie la valeur finale
```

Ceci fonctionne aussi bien pour des sommes "libres" $\sum_{i=1}^{n}\sum_{j=1}^{p} a_{i,j}$

que pour des sommes "avec contraintes" de type $\sum_{i=1}^{n}\sum_{j=i}^{n} a_{i,j}$ ou $\sum_{j=1}^{n}\sum_{i=1}^{j} a_{i,j}$ ...

Exercice 4

1. Ecrire la somme double suivante comme une "somme de somme" :

$$\sum_{\substack{1\leqslant i\leqslant 5\\ 1\leqslant j\leqslant 10}} \frac{1}{i+j} = \sum_{i=1}^{5}\left(\sum_{j=1}^{10}\frac{1}{i+j}\right)$$ (ou dans "l'autre sens" si on préfère)

2. En déduire comment compléter le programme suivant pour qu'il affiche la valeur de cette somme double.

```
S = 0

for i in range(1,6) : # i = 1,2,3,4,5

        for j in range(1,11) : # j = 1, 2, ... , 10

                S = S + 1/(i+j) # ou bien S += 1/(i+j)

print(S)
```

Valeur approchée : $\sum_{\substack{1\leqslant i\leqslant 5\\ 1\leqslant j\leqslant 10}} \frac{1}{i+j} \simeq 7,17$

Exercice 5

1. Ecrire la somme double suivante comme une "somme de somme" :

$$\sum_{1\leqslant i\leqslant j\leqslant n} 2^{i+j} = \sum_{i=1}^{n}\left(\sum_{j=i}^{n} 2^{i+j}\right) = \sum_{j=1}^{n}\left(\sum_{i=1}^{j} 2^{i+j}\right).$$

2. Définir une fonction qui prend en entrée n et renvoie la valeur de la somme double précédente.

```
def somme_double(n) :
    S = 0
    for i in range(1,n+1) : # i = 1, 2, ... , n
        for j in range(i,n+1) : # j = i, i+1, ... , n
            S = S + 1/(i+j)
    return S
```

Ou bien :

```
def somme_double(n) :
    S = 0
    for j in range(1,n+1) : # j = 1, 2, ... , n
        for i in range(1,j+1) : # j = 1, 2, ... , i
            S = S + 1/(i+j)
    return S
```

Dans les deux cas, on fait bien attention à l'indentation (marge/alignement).